TBL 3 - CORS

1. Ativar o CORS

O pacote é instalado pelo terminal com:

```
npm install cors
```

No servidor, o CORS fica habilitado antes da rota:

```
const express = require('express');
const cors = require('cors');

const app = express();

app.use(cors());

app.get('/usuarios', (req, res) => {
  res.json([{ id: 1, nome: 'Ana' }]);
});

app.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

A consulta a /usuarios retornou HTTP 200 com o seguinte resultado:

```
[{ "id": 1, "nome": "Ana" }]
```

2. Provocar e observar o erro

Sem app.use(cors()), a chamada feita pelo frontend na porta 5500 gerou um erro de CORS no navegador. Ao habilitar o middleware novamente, a consulta voltou a funcionar.

Isso acontece porque o Postman não aplica a política de mesma origem do navegador. Já o navegador verifica se a API permite que o frontend leia a resposta. Como as portas 5500 e 3000 são diferentes, as origens também são diferentes. Sem a permissão de CORS, o JavaScript não consegue acessar a resposta, mesmo que a API esteja funcionando.

3. Restringir a origem permitida

```
app.use(cors({
  origin: 'http://localhost:5500'
}));
```

Com essa configuração, o frontend em http://localhost:5500 conseguiu consultar a API. Na porta 5501, o navegador bloqueou a leitura da resposta. Qualquer mudança no protocolo, no host ou na porta representa outra origem e precisa estar permitida na configuração.

4. Testar a API com um frontend

HTML usado para consultar a rota /usuarios:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Consulta de usuários</title>
</head>
<body>
  <h1>Usuários</h1>
  <p>O resultado também aparece no console do navegador (F12).</p>
  <pre id="resultado">Consultando a API...</pre>

  <script>
    fetch('http://localhost:3000/usuarios')
      .then(res => res.json())
      .then(dados => {
        console.log(dados);
        document.getElementById('resultado').textContent =
          JSON.stringify(dados, null, 2);
      })
      .catch(erro => {
        console.error(erro);
        document.getElementById('resultado').textContent =
          'Não foi possível consultar a API.';
      });
  </script>
</body>
</html>
```

Com o backend na porta 3000 e o HTML servido na porta 5500, a chamada funcionou. O console mostrou um array com o seguinte conteúdo:

```
[
  { "id": 1, "nome": "Ana" }
]
```

5. Permitir apenas o método GET

```
app.use(cors({
  origin: 'http://localhost:5500',
  methods: ['GET']
}));
```

O objetivo é deixar a configuração de CORS de acordo com uma API de consulta. Nas respostas ao preflight, o servidor informa GET como método permitido. No teste, a consulta funcionou e uma tentativa de DELETE foi bloqueada pelo navegador.

Essa configuração não bloqueia sozinha todos os métodos no servidor. Algumas requisições, como um POST simples, não passam pelo preflight. Por isso, as operações disponíveis também precisam ser controladas nas rotas do backend.